

Universidad Tecnológica Nacional

Trabajo Practico Integrador (TPI) Programación – Gestión De Países

Sistema de Gestión, Análisis y Persistencia de Datos Geopolíticos

Informe Técnico Integrador

Asignatura: Programación I

Carrera: Tecnicatura Universitaria en Programación

Integrantes: Facundo Gómez, Ricardo Oliva

Fecha: Junio 2026

1. Introducción y Objetivos

El presente informe técnico detalla el diseño y la implementación de una aplicación de consola en Python concebida para la administración, filtrado, ordenamiento y análisis estadístico de datos demográficos y geográficos de diversos países. El sistema destaca por integrar mecanismos de persistencia física de datos a través de archivos de texto plano estructurados.

Los objetivos académicos y técnicos alcanzados en este proyecto comprenden:

- **Persistencia en Almacenamiento Secundario:** Implementar lectura y escritura de archivos en formato CSV utilizando el módulo nativo csv.
- **Robustez mediante Validaciones:** Desarrollar funciones de control de flujo capaces de interceptar errores de tipos, cadenas vacías o datos inconsistentes mediante excepciones (try-except).
- **Modularidad Dinámica:** Dividir la lógica de negocios en módulos independientes con responsabilidades únicas para garantizar un código mantenible.
- **Procesamiento de Colecciones:** Manipular colecciones en memoria (listas de diccionarios) aplicando funciones de agregación, comprensión de listas y expresiones lambda.

2. Marco Teórico

El desarrollo de este sistema de software se fundamenta en los principios de la programación estructurada y la modularización, orientados a la gestión eficiente de datos en memoria dinámica y su posterior persistencia en sistemas de almacenamiento secundario. A continuación, se detallan las bases conceptuales aplicadas en el código, debidamente respaldadas por la literatura técnica contemporánea y los requerimientos de la cátedra:

2.1 Listas: Son estructuras de datos dinámicas que permiten almacenar múltiples elementos en una misma variable. En este proyecto se utiliza una lista principal para almacenar todos los países cargados desde el archivo CSV de manera secuencial (ejemplo: `países = []`).

2.2 Diccionarios: Estructuras asociativas que almacenan información mediante pares clave-valor. Cada país se representa de forma individual mediante un diccionario con las claves definidas: 'nombre', 'poblacion', 'superficie' y 'continente', permitiendo accesos y mutaciones directas y semánticas.

2.3 Funciones: Permiten dividir el programa en módulos reutilizables con responsabilidades únicas. En este proyecto existen funciones específicas para cargar datos, buscar países, filtrar, ordenar y generar estadísticas, lo cual optimiza el mantenimiento del código.

2.4 Condicionales: Estructuras de bifurcación que permiten tomar decisiones lógicas en el flujo del programa según condiciones booleanas determinadas. Se utilizan de forma exhaustiva para validar datos entrantes y controlar el ruteo del menú principal.

2.5 Estructuras Repetitivas: Los ciclos for y while permiten repetir instrucciones de manera controlada. Se utilizan técnicamente para recorrer las colecciones de países, validar

las entradas del operador mediante bucles de reintento y mantener activo el menú principal de la aplicación.

2.6 Ordenamientos: Mecanismo computacional para facilitar la consulta y legibilidad de los datos en pantalla. Python ofrece la función de alto nivel `sorted()` la cual, combinada de forma nativa con expresiones lambda como clave de evaluación, permite ordenar registros de forma ascendente o descendente sin mutar la colección original.

2.7 Estadísticas Básicas: Algoritmos de agregación aplicados para analizar información relevante y extraer conclusiones del set de datos. El sistema calcula el país más poblado, el menos poblado, los promedios numéricos globales y las frecuencias por continente mediante el uso estratégico de `max()`, `min()`, `sum()` y `len()`.

2.8 Archivos CSV: Formato estándar (Comma Separated Values) utilizado para almacenar datos de forma tabular en texto plano. El proyecto lee y escribe en el archivo persistente `países.csv` mediante los métodos integrados del módulo nativo `csv`, garantizando que la información sobreviva al cierre del proceso de ejecución.

3. Persistencia de Datos y Modelo en Memoria

La información de los países es leída desde un archivo llamado `países.csv`. Al iniciar, los registros se transforman en estructuras jerárquicas dinámicas (una lista que contiene diccionarios), donde cada país cuenta con las claves: nombre, población, superficie y continente.

3.1. Carga Dinámica de Datos desde Archivo CSV

La función `cargar_paises` es la encargada de abrir el canal de datos y poblar la lista principal, previniendo fallos críticos de sistema si el archivo no existe o se encuentra corrupto.

```
import csv

def cargar_paises(nombre_archivo):
    paises = []
    try:
        with open(nombre_archivo, "r", encoding="utf-8") as archivo:
            lector = csv.DictReader(archivo)
            for fila in lector:
                pais = {
                    "nombre": fila["nombre"],
                    "poblacion": int(fila["poblacion"]),
                    "superficie": int(fila["superficie"]),
                    "continente": fila["continente"]
                }
                paises.append(pais)
    except FileNotFoundError:
        print("Error: No se encontró el archivo 'países.csv'")
    except ValueError as e:
        print(f"Error al convertir datos: {e}")
    return paises
```

4. Modularización de Controles e Integridad de Datos

Para evitar ingresos inválidos que alteren las operaciones lógicas y estadísticas, se desarrollaron funciones específicas de validación basadas en bucles infinitos controlados que solo se rompen ante un retorno exitoso.

4.1. Validaciones de Texto y Restricciones Geográficas

La función `validar_texto` asegura que los nombres no contengan caracteres numéricos ni queden vacíos, mientras que `validar_continente` restringe la entrada de datos a un dominio predefinido de continentes válidos.

```
def validar_texto(mensaje):
    while True:
        nombre = input(mensaje).strip().title()
        try:
            if nombre == "":
                print()
                raise ValueError("El nombre no puede estar vacío.")

            if not nombre.replace(" ", "").isalpha():
                print()
                raise ValueError("El nombre debe contener solo letras.")

            return nombre.strip()

        except ValueError as e:
            print("Error:", e)

def validar_continente():
    continentes_validos = ["Africa", "America", "Asia", "Europa", "Oceania"]
    while True:
        continente = input("Ingrese el continente del país:
").strip().capitalize()
        try:
            if continente not in continentes_validos:
                print()
                raise ValueError(f"El continente debe ser uno de los
siguientes: {' '.join(continentes_validos)}.")
            return continente
        except ValueError as e:
            print("Error:", e)
```

5. Implementación de Módulos Operacionales y de Consulta

5.1. Análisis Estadístico Basado en Funciones de Agregación

La función `mostrar_estadisticas` explota el uso de las funciones integradas de Python (`max`, `min`, `sum`) combinadas con funciones anónimas `lambda` y comprensiones de listas para determinar de forma eficiente los extremos demográficos y los promedios globales del repositorio, además de mapear la distribución de países utilizando un diccionario de frecuencias.

```
def mostrar_estadisticas(paises):
    print("\n--- ESTADÍSTICAS ---")

    pais_mas_poblado = max(paises, key=lambda x: x["poblacion"])
    pais_menos_poblado = min(paises, key=lambda x: x["poblacion"])
    promedio_poblacion = sum(pais["poblacion"] for pais in paises) /
len(paises)
    promedio_superficie = sum(pais["superficie"] for pais in paises) /
len(paises)

    continentes = {}
    for pais in paises:
        continente = pais["continente"]
        if continente in continentes:
            continentes[continente] += 1
        else:
            continentes[continente] = 1

    print(f"País más poblado: {pais_mas_poblado['nombre']} con
{pais_mas_poblado['poblacion']} habitantes.")
    print(f"País menos poblado: {pais_menos_poblado['nombre']} con
{pais_menos_poblado['poblacion']} habitantes.")
    print(f"Promedio de población: {promedio_poblacion:.2f} habitantes.")
    print(f"Promedio de superficie: {promedio_superficie:.2f} km².")
    print("Número de países por continente:")
    for continente, cantidad in continentes.items():
        print(f"{continente}: {cantidad} país(es)")
```

5.2. Algoritmos de Filtrado por Rangos Acotados

El filtrado por variables cuantitativas (como la población) evalúa las fronteras numéricas mediante condiciones booleanas compuestas, aislando los registros que coinciden con los rangos mínimos y máximos solicitados al operador.

```
def filtrar_por_poblacion(paises):
    poblacion_minima = validar_entero("Ingrese la población mínima: ",1)
    poblacion_maxima = validar_entero("Ingrese la población máxima: ",1)

    encontrados = []
```

```

for pais in paises:
    if poblacion_minima <= pais["poblacion"] <= poblacion_maxima:
        encontrados.append(pais)

while poblacion_minima > poblacion_maxima:
    print("Error: La población mínima no puede ser mayor que la
población máxima.")
    poblacion_minima = validar_entero("Ingrese la población mínima: ",1)
    poblacion_maxima = validar_entero("Ingrese la población máxima: ",1)

if len(encontrados) == 0:
    print("No se encontraron países dentro del rango de población
especificado.")
else:
    print(f"Se encontraron {len(encontrados)} país(es) dentro del rango
de población especificado:")
    mostrar_paises(encontrados)

```

6. Arquitectura del Flujo General (Menú Principal)

El sistema unifica sus hilos de ejecución en una interfaz cíclica controlada por la función main y regulada por la lectura de opciones de la función menu. La salida del programa ejecuta una llamada obligatoria a guardar_paises para volcar los cambios de la memoria física de regreso al archivo persistente.

```

def menu():
    while True:
        try:
            print("\n--- MENÚ PRINCIPAL ---")
            print("1. Mostrar todos los países")
            print("2. Agregar un nuevo país")
            print("3. Actualizar población y superficie de un país
existente")
            print("4. Buscar un país por nombre")
            print("5. Filtrar países")
            print("6. Ordenar países")
            print("7. Mostrar estadísticas")
            print("8. Guardar y salir")

            opcion = input("Seleccione una opción (1-8): \n").strip()
            if opcion not in [str(i) for i in range(1, 9)]:
                raise ValueError("Opción no válida. Por favor, seleccione
una opción entre 1 y 8.")
            return opcion
        except ValueError as e:
            if "invalid literal" in str(e):
                print("Error: Debe ingresar un número.")
            else:
                print("Error:", e)

```

7. Conclusión

La aplicación diseñada responde estrictamente a las necesidades de procesamiento ordenado de estructuras homogeneizadas en Python. Al delegar las tareas operativas en funciones específicas y centralizar la persistencia mediante el estándar CSV, se consolida una base de software modular, estable y resistente a fallos de tipado o ingresos erróneos por parte del usuario. La implementación permitió aplicar con solidez los conceptos fundamentales de Programación I: estructuras de datos dinámicas, modularización mediante funciones, validaciones rigurosas y manejo de archivos de almacenamiento secundario.

8. Bibliografía y Fuentes de Referencia

1. Cairó, O. (2005). Metodología de la Programación: Algoritmos, diagramas de flujo y programas (3ª ed.). Alfaomega.
2. Python Software Foundation. (2026). The Python Standard Library: csv — CSV File Reading and Writing. Disponible en la documentación oficial de Python.
3. Van Rossum, G., & Drake, F. L. (2009). An Introduction to Python. Network Theory Ltd.